Analisis Struktur Data dan Algoritma

Project DepSolve CLI

---

1. Pendahuluan

DepSolve CLI adalah aplikasi command-line berbasis Python yang digunakan untuk mensimulasikan cara kerja package manager modern dalam mengelola package dan dependency. Aplikasi ini dirancang bukan sebagai sistem informasi biasa, melainkan sebagai tools edukatif yang menekankan penerapan struktur data dan algoritma secara nyata.

Pada aplikasi ini, setiap package dapat memiliki ketergantungan terhadap package lain. Hubungan tersebut membentuk sebuah graph yang harus dianalisis untuk menentukan urutan instalasi, mendeteksi circular dependency, menampilkan dependency tree, serta memvalidasi kompatibilitas dependency.

Karena itu, inti dari project ini terletak pada implementasi:

- graph,
- hash map,
- stack,
- queue,
- traversal algoritma,
- topological sorting,
- dan cycle detection.

Dokumen ini menjelaskan struktur data dan algoritma yang digunakan pada setiap fitur utama DepSolve CLI.

---

2. Tujuan Analisis

Analisis ini dibuat untuk:

1. Menjelaskan struktur data yang digunakan dalam setiap fitur aplikasi.
2. Menjelaskan algoritma yang digunakan untuk memproses dependency package.
3. Menunjukkan hubungan antara kebutuhan fitur dengan pemilihan struktur data.
4. Menjadi dasar implementasi sistem pada tahap coding.
5. Menjadi materi pendukung penjelasan pada laporan dan presentasi UAS.

---

3. Gambaran Umum Sistem

DepSolve CLI bekerja dengan cara membaca data package dan dependency dari database MySQL, lalu membangun representasi graph di dalam program Python. Graph ini digunakan untuk menjalankan berbagai proses seperti:

- pencarian package,
- penelusuran dependency,
- pengecekan cycle,
- penyusunan urutan instalasi,
- dan pengecekan konflik dependency.

Secara umum, alur kerja sistem adalah:

1. User menjalankan perintah CLI.
2. Sistem membaca request dari terminal.
3. Program mengambil data package dari database.
4. Data dependency diubah menjadi graph.
5. Algoritma dijalankan sesuai kebutuhan fitur.
6. Hasil diproses dan ditampilkan ke terminal.
7. Jika ada perubahan status package, data diperbarui ke database.

---

4. Struktur Data Utama yang Digunakan

4.1 Graph

Graph adalah struktur data utama dalam project ini karena hubungan antar package dan dependency secara alami dapat dimodelkan sebagai graph.

Dalam konteks DepSolve CLI:

- setiap package adalah node/vertex,
- setiap dependency adalah edge,
- arah edge menunjukkan ketergantungan dari package utama ke package yang dibutuhkan.

Contoh:

- Flask → Werkzeug
- Flask → Jinja2

Artinya Flask membutuhkan Werkzeug dan Jinja2 agar dapat diinstal atau dijalankan dengan benar.

Alasan penggunaan graph

Graph dipilih karena:

- merepresentasikan relasi dependency secara natural,
- mendukung traversal antar node,
- cocok untuk deteksi circular dependency,
- cocok untuk topological sorting.

Representasi graph yang digunakan

Graph dapat disimpan sebagai adjacency list, karena:

- lebih hemat memori dibanding adjacency matrix,
- lebih efisien untuk graph yang relatif jarang,
- lebih cocok untuk traversal dependency.

Contoh adjacency list:

```
Flask -> [Werkzeug, Jinja2]
Django -> [pytz, sqlparse, asgiref]
```

4.2 Hash Map (Dictionary)

Hash map digunakan untuk melakukan pencarian data secara cepat. Dalam Python, struktur ini dapat diimplementasikan menggunakan `dict`.

Fungsi hash map dalam project

- menyimpan data package berdasarkan nama,
- mempercepat pencarian package,
- menyimpan adjacency list graph,
- menyimpan cache dependency,
- memeriksa apakah package sudah pernah diproses.

Alasan penggunaan hash map

Hash map memiliki rata-rata kompleksitas akses `O(1)`, sehingga sangat cocok untuk:

- validasi package,
- pencarian dependency,
- mencegah proses traversal berulang pada node yang sama.

Contoh:

```
package_map = {
    "flask": {"version": "3.0.0", "dependencies": ["werkzeug", "jinja2"]},
    "werkzeug": {"version": "3.0.1", "dependencies": []}
}
```

4.3 Stack

Stack digunakan pada algoritma DFS dan cycle detection.

Fungsi stack dalam project

- menyimpan node yang sedang diproses,
- mendeteksi node yang sedang berada dalam recursion path,
- membantu mengetahui apakah terjadi cycle.

Alasan penggunaan stack

Struktur stack bekerja dengan prinsip LIFO (Last In, First Out), sehingga cocok untuk:

- penelusuran kedalaman graph,
- penyimpanan state traversal,
- identifikasi dependency yang membentuk loop.

Dalam implementasi DFS recursion, call stack juga berperan sebagai stack logis untuk proses traversal.

4.4 Queue

Queue digunakan pada proses yang membutuhkan penelusuran level-by-level.

Fungsi queue dalam project

- membantu BFS traversal,
- mendukung proses instalasi bertahap,
- dapat digunakan untuk antrian package yang siap diproses.

Alasan penggunaan queue

Queue bekerja dengan prinsip FIFO (First In, First Out), sehingga cocok untuk:

- penelusuran dependency secara berlapis,
- proses instalasi yang mengikuti urutan dependensi,
- simulasi antrian task.

4.5 Tree

Tree sebenarnya merupakan bentuk visual dari graph dependency untuk kebutuhan tampilan dependency tree.

Fungsi tree dalam project

- menampilkan struktur dependency secara hierarkis,
- memudahkan pengguna memahami relasi package,
- menjadi representasi output dari proses DFS.

Walaupun data dasarnya berupa graph, hasil tampilannya dapat dibentuk seperti tree ketika menampilkan dependency satu package tertentu.

---

5. Algoritma Utama yang Digunakan

5.1 Depth First Search (DFS)

DFS digunakan untuk menelusuri dependency dari sebuah package hingga ke dependency terdalam.

Fungsi DFS dalam project

- menampilkan dependency tree,
- mengecek dependency secara mendalam,
- membantu cycle detection,
- menelusuri semua dependency sebelum instalasi.

Cara kerja DFS

DFS memulai penelusuran dari package target, lalu masuk ke dependency pertama, kemudian dependency dari dependency tersebut, dan seterusnya sampai tidak ada dependency lagi.

Contoh:

```
Flask -> Werkzeug -> MarkupSafe
```

DFS akan masuk dari Flask ke Werkzeug, lalu dari Werkzeug ke dependency yang lebih dalam jika ada.

Kelebihan DFS

- cocok untuk penelusuran dependency yang mendalam,
- efisien untuk graph relasi package,
- mudah digunakan untuk deteksi cycle.

Kompleksitas

- Time Complexity: `O(V + E)`
- Space Complexity: `O(V)`

5.2 Breadth First Search (BFS)

BFS digunakan untuk menelusuri dependency berdasarkan level.

Fungsi BFS dalam project

- eksplorasi dependency secara bertahap,
- alternatif untuk analisis dependency level-by-level,
- membantu pemrosesan antrian package.

Cara kerja BFS

BFS memproses package pada level paling atas terlebih dahulu, lalu turun ke dependency berikutnya.

Contoh:

```
Level 0: Flask
Level 1: Werkzeug, Jinja2
Level 2: dependency dari Werkzeug atau Jinja2
```

Kelebihan BFS

- cocok untuk melihat dependency per level,

- memudahkan visualisasi urutan relatif antar node,
- dapat digunakan sebagai dasar antrian instalasi.

Kompleksitas

- Time Complexity: `O(V + E)`
- Space Complexity: `O(V)`

5.3 Topological Sort

Topological sort adalah algoritma penting untuk menentukan urutan instalasi package yang benar.

Fungsi topological sort dalam project

- menentukan urutan install dependency,
- memastikan dependency diinstal sebelum package utama,
- menghasilkan urutan yang valid dalam graph berarah asiklik.

Contoh

Jika:

- Flask bergantung pada Werkzeug,
- Flask bergantung pada Jinja2,

maka urutan instalasi yang benar adalah:

1. Werkzeug
2. Jinja2
3. Flask

Alasan penggunaan topological sort

Dalam dependency management, package tidak boleh diinstal sebelum dependency-nya tersedia. Topological sort memastikan urutan tersebut benar.

Kompleksitas

- Time Complexity: `O(V + E)`
- Space Complexity: `O(V)`

5.4 Cycle Detection

Cycle detection digunakan untuk mendeteksi circular dependency.

Fungsi cycle detection dalam project

- mencegah dependency loop,
- memastikan graph tetap valid,
- menghindari proses instalasi yang tidak pernah selesai.

Contoh circular dependency

```
A -> B -> C -> A
```

Jika pola ini terjadi, sistem harus menolak proses instalasi karena dependency saling bergantung dalam lingkaran.

Metode yang digunakan

Cycle detection dapat dilakukan menggunakan DFS dengan bantuan:

- visited set,
- recursion stack.

Jika saat DFS ditemukan node yang sudah ada di recursion stack, maka cycle terdeteksi.

Kompleksitas

- Time Complexity: `O(V + E)`
- Space Complexity: `O(V)`

---

6. Mapping Fitur ke Struktur Data dan Algoritma

Fitur	Struktur Data	Algoritma	Tujuan
Install Package	Graph, Hash Map, Queue	DFS, Topological Sort	Menginstal package beserta dependency-nya
Remove Package	Hash Map, Graph	Validasi relasi dependency	Menentukan apakah package aman dihapus
Dependency Tree	Graph, Tree, Stack	DFS	Menampilkan dependency secara hierarkis
Conflict Detection	Graph, Hash Map	Traversal + Validasi versi	Mendeteksi konflik dependency
Circular Dependency Detection	Graph, Stack	DFS Cycle Detection	Mendeteksi loop pada dependency
Installed Package List	Hash Map, List	Query database	Menampilkan package yang sudah diinstal
Version Validation	Hash Map	Perbandingan versi	Memastikan dependency kompatibel

7. Analisis Struktur Data per Fitur

7.1 Fitur Install Package

Fitur install package adalah fitur inti dalam DepSolve CLI.

Proses yang dilakukan

1. User memasukkan nama package.
2. Sistem mencari package di database.
3. Sistem membangun dependency graph.
4. Sistem mengecek apakah ada circular dependency.
5. Sistem melakukan topological sort.
6. Dependency diinstal terlebih dahulu.
7. Package utama diinstal terakhir.

8. Hasil instalasi disimpan ke database.

Struktur data yang dipakai

- Graph untuk hubungan dependency.
- Hash map untuk lookup package.
- Stack untuk DFS dan cycle detection.
- Queue dapat digunakan jika instalasi dilakukan bertahap.

Algoritma yang dipakai

- DFS untuk traversal dependency.
- Topological sort untuk urutan instalasi.
- Cycle detection untuk validasi graph.

7.2 Fitur Remove Package

Fitur remove package digunakan untuk menghapus package yang sudah diinstal.

Proses yang dilakukan

1. User memilih package yang ingin dihapus.
2. Sistem mengecek apakah ada package lain yang bergantung padanya.
3. Jika masih digunakan, penghapusan ditolak.
4. Jika tidak digunakan, package dihapus dari daftar installed.
5. Database diperbarui.

Struktur data yang dipakai

- Graph untuk mengecek relasi dependency.
- Hash map untuk pencarian cepat.
- List/array untuk menyimpan daftar dependency terkait.

Algoritma yang dipakai

- Traversal relasi dependency.
- Validasi dependents sebelum penghapusan.

7.3 Fitur Dependency Tree Visualization

Fitur ini digunakan untuk menampilkan hubungan dependency dalam bentuk tree di terminal.

Proses yang dilakukan

1. User memilih package.
2. Sistem mencari dependency package tersebut.
3. Sistem melakukan DFS.
4. Setiap node ditampilkan dengan indentasi atau simbol tree.
5. Hasil ditampilkan sebagai struktur hierarkis.

Struktur data yang dipakai

- Graph sebagai sumber data.
- Stack untuk DFS.
- Tree sebagai bentuk output visual.

Algoritma yang dipakai

- DFS traversal.

7.4 Fitur Conflict Detection

Fitur ini digunakan untuk mendeteksi konflik versi atau dependency yang tidak cocok.

Proses yang dilakukan

1. Sistem membaca semua dependency package.
2. Sistem membandingkan requirement versi antar package.
3. Jika ditemukan versi tidak kompatibel, konflik dicatat.

4. Hasil konflik ditampilkan ke user.

Struktur data yang dipakai

- Hash map untuk menyimpan versi package.
- Graph untuk melihat relasi dependency.
- List untuk menampung konflik yang ditemukan.

Algoritma yang dipakai

- Traversal graph.
- Perbandingan versi.
- Validasi constraint dependency.

7.5 Fitur Circular Dependency Detection

Fitur ini sangat penting untuk memastikan graph dependency valid.

Proses yang dilakukan

1. Sistem memulai DFS dari node package.
2. Node yang sedang aktif dimasukkan ke recursion stack.
3. Jika DFS menemukan node yang sudah ada di stack, berarti ada cycle.
4. Sistem menghentikan proses dan memberi notifikasi error.

Struktur data yang dipakai

- Graph.
- Stack.
- Hash map untuk visited state.

Algoritma yang dipakai

- DFS Cycle Detection.

7.6 Fitur Installed Package List

Fitur ini menampilkan daftar package yang sudah terinstal.

Proses yang dilakukan

1. Sistem membaca data dari tabel installed_packages.
2. Sistem menampilkan data dalam bentuk daftar.
3. User dapat melihat package beserta waktu instalasi.

Struktur data yang dipakai

- List.
- Hash map untuk akses data package.

Algoritma yang dipakai

- Query database dan formatting output.

7.7 Fitur Version Validation

Fitur ini digunakan untuk memastikan dependency cocok dengan versi yang dibutuhkan.

Proses yang dilakukan

1. Sistem membaca versi package dan dependency minimum.
2. Sistem membandingkan versi aktual dengan requirement.
3. Jika versi lebih kecil dari minimum, dependency dianggap tidak valid.
4. Jika valid, proses dapat dilanjutkan.

Struktur data yang dipakai

- Hash map.
- String/array untuk parsing versi.

Algoritma yang dipakai

- Perbandingan versi.

- Parsing semantic-like version sederhana.

---

8. Rancangan Data Internal di Dalam Program

Walaupun data utama disimpan di MySQL, saat program berjalan data akan dimuat ke dalam struktur data internal agar proses lebih cepat.

Contoh representasi internal:

```
package_graph = {
    "flask": ["werkzeug", "jinja2"],
    "werkzeug": [],
    "jinja2": []
}
```

Atau:

```
package_versions = {
    "flask": "3.0.0",
    "werkzeug": "3.0.1",
    "jinja2": "3.1.0"
}
```

Struktur seperti ini membuat:

- traversal lebih cepat,
- pengecekan dependency lebih sederhana,
- proses topological sort lebih efisien.

---

9. Hubungan Database dengan Struktur Data

Database MySQL berfungsi sebagai penyimpanan permanen, sedangkan struktur data di Python berfungsi sebagai penyimpanan sementara selama proses berjalan.

Tabel `packages`

Menyimpan data package utama.

Tabel `dependencies`

Menyimpan relasi antar package sehingga graph dapat dibangun.

Tabel `installed_packages`

Menyimpan data package yang telah terinstal.

Hubungan ini penting karena:

- database menyediakan data mentah,
- program mengubah data tersebut menjadi graph,
- algoritma dijalankan pada graph tersebut,
- hasil disimpan kembali ke database jika ada perubahan.

---

10. Analisis Kompleksitas Umum

Berikut estimasi kompleksitas dari proses utama:

Proses	Time Complexity	Space Complexity
Lookup package	`O(1)` rata-rata	`O(1)`
DFS traversal	`O(V + E)`	`O(V)`
BFS traversal	`O(V + E)`	`O(V)`
Topological sort	`O(V + E)`	`O(V)`
Cycle detection	`O(V + E)`	`O(V)`
Conflict validation	`O(V + E)`	`O(V)`

Keterangan:

- `V` = jumlah node/package
- `E` = jumlah edge/dependency

---

11. Alasan Pemilihan Struktur Data

Graph

Dipilih karena dependency package memang membentuk relasi antar node.

Hash Map

Dipilih karena pencarian package harus cepat dan efisien.

Stack

Dipilih karena DFS dan cycle detection membutuhkan jejak proses yang sesuai dengan prinsip LIFO.

Queue

Dipilih karena BFS dan antrian proses instalasi membutuhkan urutan FIFO.

Tree

Dipilih untuk menampilkan hasil dependency dalam bentuk yang mudah dipahami user.

---

12. Kaitan dengan Mata Kuliah Struktur Data

Project DepSolve CLI sangat relevan dengan mata kuliah Struktur Data karena mencakup penerapan:

- graph,
- traversal graph,
- DFS dan BFS,
- stack,
- queue,
- hash map,
- topological sort,
- dan cycle detection.

Project ini tidak hanya menampilkan teori struktur data, tetapi juga menerapkannya dalam kasus nyata yang mirip dengan sistem package manager modern.

Dengan demikian, project ini dapat menjadi bukti bahwa konsep struktur data dan algoritma dapat digunakan untuk menyelesaikan masalah praktis di dunia software

engineering.

---

13. Kesimpulan

DepSolve CLI adalah aplikasi command-line yang berfokus pada simulasi package dependency resolver. Dalam implementasinya, aplikasi ini sangat bergantung pada struktur data graph untuk merepresentasikan hubungan dependency antar package.

Algoritma utama yang digunakan meliputi DFS, BFS, topological sort, dan cycle detection. Hash map digunakan untuk mempercepat pencarian, stack digunakan dalam traversal dan deteksi cycle, sedangkan queue digunakan untuk proses berurutan.

Secara keseluruhan, project ini sangat sesuai sebagai proyek UAS Struktur Data karena:

- memiliki masalah nyata yang kompleks,
- menggunakan struktur data yang relevan,
- menerapkan algoritma penting,
- dan tetap realistis untuk diimplementasikan dalam bentuk CLI berbasis Python dengan database MySQL.

Kalau mau, next aku bisa lanjut bikin versi **lebih formal dan rapi untuk laporan akademik**, atau langsung bikin **flowchart install package** supaya dokumennya lengkap.